Datamecum

Modelos fundacionales en visión artificial

Tema 2: Hugging Face

Día 1 del curso

Web del curso	datamecum.sergiovillanuevalopez.com
Práctica	Cuaderno 2 en Colab (2_HF_Intro.ipynb)
Profesor	Sergio Villanueva · Datamecum

2. Hugging Face

Buscar modelos, revisar su licencia y usarlos.

Hugging Face es hoy el **repositorio de referencia** para modelos de IA, datasets, demos interactivas y herramientas de desarrollo. Desde la misma web se busca un modelo, se prueba y se descarga.



Qué es Hugging Face

Hugging Face reúne **cuatro piezas que se usan juntas**.

Model Hub

Catálogo con más de tres millones de modelos preentrenados: visión, lenguaje, audio y multimodales.

Datasets

Más de un millón de datasets públicos para entrenar y evaluar, con una API común de carga y procesamiento.

Spaces

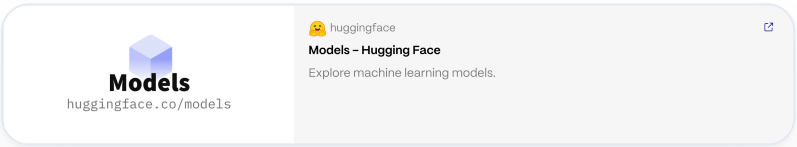
Demos web interactivas, casi siempre con Gradio, que Hugging Face aloja y ejecuta. Pruebas un modelo en segundos sin instalar nada.

Transformers

Librería Python con una misma API de carga, inferencia y fine-tuning para cientos de arquitecturas distintas.

El flujo típico

1. **Buscar modelo**
2. **Probar en Space**
3. **Descargar con Transformers**
4. **Integrar en tu aplicación**



huggingface.co/models

Cómo buscar un modelo

El buscador del Hub tiene estos **filtros**:

- **Tarea:** object-detection, image-classification, text-generation, etc.
- **Licencia:** Apache-2.0, MIT, CreativeML, Research-only.
- **Popularidad:** número de descargas, likes, fecha de última actualización.

La ficha del modelo (model card) es lo primero que hay que leer. Una ficha completa trae:

Descripción y uso

Para qué fue entrenado, qué tareas resuelve, cómo usarlo con código de ejemplo.

Limitaciones

Sesgos conocidos, casos donde falla, advertencias de seguridad y aspectos éticos.

Licencia

Términos legales de uso: ¿puedes usarlo comercialmente? ¿Debes atribuir autoría?

Archivos

Configuración JSON, pesos del modelo (.safetensors preferido), tokenizers, ejemplos.

Métricas

Resultados en benchmarks estándar, comparación con otros modelos.

Las descargas y los likes sirven para descubrir candidatos, no para decidir cuál funciona mejor en tu dominio. La fecha de última actualización avisa de modelos abandonados.

La ficha del modelo

La Model Card contesta si el modelo sirve para tu caso antes de gastar tiempo.

Buscas un clasificador para imágenes médicas. ¿Qué haces primero?

- A Elegir el modelo con más descargas.
- B Filtrar por tarea y leer la ficha del modelo pensando en tu caso y en la licencia.
- C Descargar varios y quedarte con el más pequeño.

Respuesta B. Exacto: la tarea acota la búsqueda y la ficha dice para qué se entrenó y con qué condiciones.

Un dominio sensible obliga a revisar para qué se entrenó el modelo, con qué datos y qué limitaciones declara. El número de descargas no responde a ninguna de esas preguntas.

Licencias

La licencia dice **para qué puedes usar el modelo**: en un producto, en un servicio web o solo para investigar.

Comercial OK

- **Apache-2.0:** muy permisiva, ideal para empresas.
- **MIT:** similar a Apache, aún más simple.
- **BSD:** variantes 2-clause y 3-clause, ambas permisivas.

Con restricciones

- **OpenRAIL/CreativeML:** uso comercial OK, pero con cláusulas de uso responsable.
- **CC BY-SA:** requiere compartir derivados bajo la misma licencia.

No comercial

- **Research Only:** solo investigación académica.
- **CC BY-NC:** prohibido uso comercial.

Caso especial AGPL: algunas familias YOLO populares usan AGPL, que **obliga a publicar el código de la aplicación que lo incorpora** si integras el modelo en una aplicación web o servicio. Si no quieres publicar tu código, busca alternativas con Apache-2.0 como RT-DETR o RF-DETR.

Las condiciones se revisan por artefacto y por versión: el repositorio de código y los pesos publicados pueden llevar licencias distintas, y ambas pueden cambiar entre versiones. Esta clasificación orienta la búsqueda; la decisión final se toma leyendo la licencia vigente del artefacto concreto.

Licencia según el uso

La misma licencia cambia de respuesta según lo que vayas a hacer con el modelo.

AGPL

Servicio web o SaaS

Permitido con condiciones

La cláusula de red obliga a publicar el código de la aplicación que lo incorpora aunque solo la ofrezcas por web. Alternativas Apache-2.0: RT-DETR o RF-DETR.

Orientación para buscar, no asesoramiento legal. La licencia real se lee en el artefacto y la versión que vayas a usar.

La tabla recoge lo que explica la diapositiva anterior. Los tres casos que más sorprenden: AGPL no obliga a nada mientras no distribuyas ni sirvas el modelo, pero sí en cuanto lo expones por web; una licencia no comercial tampoco vale para un prototipo interno de empresa, porque el propósito ya es comercial; y CC BY-SA permite vender, pero obliga a publicar los derivados con la misma licencia.

Licencia del código y de los pesos

Un repositorio permisivo no garantiza que los pesos lo sean.

El código de un repositorio tiene licencia Apache, pero los pesos llevan otra. ¿Qué revisas?

- A Solo la licencia del código.
- B Nada, si se puede descargar gratis.
- C La licencia del código, la de los pesos y la de la versión que vas a usar.

Respuesta C. Exacto: cada pieza puede tener sus propias condiciones.

Conviene revisar la licencia del repositorio, la del checkpoint concreto y la de los datos de entrenamiento cuando se declara, además de la versión a la que se aplican.

Qué pasa al cargar un modelo

Lo que hace la librería Transformers al cargar un modelo:

1 **Hub remoto**

El modelo vive en Hugging Face Hub identificado por su nombre (ej: "facebook/detr-resnet-50").

2 **Descarga a caché**

Primera ejecución: descarga config.json, pesos .safetensors, y metadatos a ~/.cache/huggingface/

3 **Carga en memoria**

El modelo se instancia en RAM/VRAM según el dispositivo (CPU/GPU) que especifiques.

4 **Inferencia**

Procesas inputs y obtienes predicciones. Puedes usar GPU y mixed precision para acelerar.

Buenas prácticas

- **Seguridad:** prefiere .safetensors sobre .bin; los .bin usan pickle, que puede ejecutar código al cargarse.
- **Offline:** predescarga con snapshot_download, controla cache_dir.
- **Rendimiento:** usa float16 y device="cuda" cuando tengas GPU.
- **Pipelines:** para pruebas rápidas, usa pipeline("task"); en producción, clases específicas dan más control.

Qué hace un pipeline

Con una sola llamada, dando la tarea y el nombre del modelo:

1. **Descarga**

   Baja del Hub el modelo y lo que necesita para preparar la imagen.

2. **Ejecuta**

   Prepara la imagen y la pasa por el modelo.

3. **Devuelve**

   El resultado ya legible: etiqueta, confianza y, si es un detector, la caja.

> El código está en el cuaderno 2: son tres líneas.

El identificador del modelo determina qué arquitectura y qué preprocesador se instancian. Para reproducibilidad conviene fijar también la revisión. En producción, las clases específicas permiten controlar batching, dispositivo, precisión y formato de salida.

Qué te ahorra un pipeline

Conviene saber qué pasos quedan ocultos antes de llevarlo a producción.

¿Qué simplifica un pipeline de Transformers?

- A La preparación de entrada y la ejecución de una tarea común.
- B La necesidad de conocer la licencia y la model card.
- C El entrenamiento de cualquier modelo sin datos.

Respuesta A. Exacto: el pipeline agrupa pasos frecuentes para una primera prueba.

El pipeline reúne preprocesado, inferencia y posprocesado para una tarea común. No elige el modelo por ti ni garantiza el mejor rendimiento en tu dominio.

Memoria de GPU

Primera pregunta al elegir modelo, y se responde con una multiplicación.

Parámetros del modelo **300 millones**

Modelos de referencia (pulsa uno):

ResNet-50 | ViT-B/16 | SAM 2.1 large | Qwen3.5-2B | DINOv3 7B | Llama 3.1 70B | DeepSeek-V3 | Kimi K2

Formato de los números

float16 o bfloat16 · 2 bytes

Memoria estimada para inferencia

0,8 GB

Solo los pesos

0,6 GB

Del orden de SAM 2.1 large (segmentar, 224 millones).

Portátil · 8 GB | T4 (Colab gratis) · 15 GB | L4 · 24 GB | A100 · 40 GB | H100 · 80 GB

Cabe en 5 de las 5 tarjetas de la lista, contando solo inferencia.

Pesos por precisión más un 35 % de margen para activaciones y fragmentación, con lotes pequeños. Entrenar necesita bastante más. Los chats como ChatGPT no publican su tamaño; se calcula que pasan del billón de parámetros.

La cuenta es directa: parámetros por bytes de cada precisión. En float32 cada parámetro ocupa 4 bytes, en float16 dos, y en FP8 o int8 uno. Una ResNet-50 de clasificación son 25 millones de parámetros y cabe en cualquier sitio; un chat abierto de 70 mil millones ya no entra en una sola H100 en float16; uno de un billón necesita decenas de tarjetas.

Modelos de visión para empezar

Filtra por tarea y por licencia, que son los dos criterios que descartan antes.

`PekingU/rtdetr_r50vd`	Detección COCO, tiempo real	**Apache-2.0**	~42 M
`facebook/detr-resnet-50`	El DETR original, de referencia	**Apache-2.0**	~41 M
`hustvl/yolos-small`	Detección con ViT puro	**Apache-2.0**	~31 M
`IDEA-Research/grounding-dino-base`	Cajas a partir de una frase	**Apache-2.0**	~233 M
`google/owlv2-base-patch16`	Vocabulario abierto por texto	**Apache-2.0**	~154 M
`facebook/sam2.1-hiera-large`	Segmentación con prompts, imagen y vídeo	**Apache-2.0**	~224 M
`facebook/dinov2-base`	Features densas sin etiquetas	**Apache-2.0**	~86 M
`openai/clip-vit-base-patch32`	Imagen y texto en un espacio común	**MIT**	~151 M
`briaai/RMBG-1.4`	Recorte de fondo con canal alfa	**No comercial**	~44 M

Lista consultada el 17 de septiembre de 2026 y pensada como punto de partida, no como ranking. Los tamaños son aproximados y la licencia debe confirmarse en la ficha del modelo y de sus pesos antes de usarlo, porque puede cambiar entre versiones. Fíjate en el caso de RMBG-1.4: excelente para recortar fondo, pero su licencia no permite uso comercial sin acuerdo.

Spaces: demos en el navegador

Un Space es una demo web del modelo, ya montada. Subes una foto y ves la salida, sin instalar nada.

Quitar el fondo

Sube una foto de producto y descarga el recorte con transparencia. Es el modelo del tema 6 puesto a trabajar.

Space de BRIA RMBG 1.4

Profundidad de una sola foto

Sube cualquier foto y mira el mapa de distancias. Prueba con espejos o cristales, que hacían fallar a la versión anterior, y comprueba si acierta.

Space de Depth Anything V2

Detectar escribiendo la clase

Página de la tarea, con los modelos, los datasets y los Spaces que están vivos hoy. Escribe "una caja de cartón" y mira qué encuentra.

Tarea: detección zero-shot

Segmentar

La misma idea para máscaras: modelos de segmentación semántica, de instancias y promptable, con demos que puedes abrir al momento.

Tarea: segmentación de imagen

Puntos clave

Pose de personas y puntos de interés para matching, con más de 400 modelos publicados y sus demos.

Tarea: detección de puntos clave

Preguntar sobre una imagen

Los modelos que reciben imagen y texto y responden. Sube una foto de una etiqueta y pídele los campos en JSON.

Tarea: imagen y texto a texto

Dos avisos antes de subir nada. Un Space público procesa tu imagen en el servidor de quien lo publicó, así que ahí no van fotos de cliente, de personas identificables ni de piezas bajo acuerdo de confidencialidad. Y los Spaces se duermen, se rompen y desaparecen: por eso los enlaces de arriba apuntan a la página de la tarea, que sigue viva aunque una demo concreta caiga.

Un Space es un repositorio con una aplicación (Gradio, Docker o HTML estático) que Hugging Face construye y ejecuta. En el hardware gratuito, CPU Basic, se duerme tras un rato sin visitas, de ahí que a veces tarde en arrancar o falle al construirse. Con el botón de duplicar tienes tu propia copia para cambiarle el modelo, la forma más rápida de enseñar un resultado a alguien que no programa. Ten en cuenta que crear un Space con Gradio o Docker pide hoy un plan de pago, salvo hasta dos Spaces de Gradio en ZeroGPU para cuentas gratuitas; los de HTML estático son gratis. Para un piloto interno conviene duplicarlo en privado en lugar de usar el público.

Antes de quedarte con un modelo del Hub

Siete cosas que comprobar antes de elegirlo.

- [ ] **La tarea coincide**
  La etiqueta del Hub dice lo que necesitas, no algo parecido.
- [ ] **La licencia permite tu uso**
  Comprobada en el repositorio y en los pesos, para la versión que vas a usar.
- [ ] **La ficha declara limitaciones**
  Sesgos, casos donde falla, datos de entrenamiento. Si no dice nada, desconfía.
- [ ] **Cabe en tu hardware**
  Con la precisión que vas a usar y con margen para el lote.
- [ ] **Formato de pesos seguro**
  .safetensors antes que .bin.
- [ ] **Revisión anotada**
  Identificador y revisión guardados para poder reproducir.
- [ ] **Probado con tus datos**
  Una decena de imágenes tuyas dicen más que cualquier métrica publicada.

Cuaderno 2: Hugging Face

Los ejercicios están en el cuaderno. El extra, si da tiempo, lo hacemos juntos; si no, queda para casa.

1. Cargar un clasificador y usarlo
2. Un catálogo de clásicos: ResNet, ViT, ConvNeXt...
3. El RF-DETR del cuaderno 1, desde el Hub
4. Guardar un modelo en una carpeta
5. Ejercicio: otro clasificador
6. Extra: otro detector del Hub

Abrir en Colab ↗

En Colab: Archivo, Guardar una copia en Drive, y Entorno de ejecución, GPU T4.